Where QA Copilot's capacity limits come from, and how to raise them

An analysis of Figma, the AI layer and QA Copilot's own processing limits, based on the current code, measurements from the Ascendion scans (22 and 24 October 2026) and the providers' official pages read on 24 October 2026.

`confirmed` official documentation `observed` measured on our installation `estimate` derived, assumptions stated

`our app` a limit set in QA Copilot's code or configuration

1. Executive summary

The AI context window is not the bottleneck

QA Copilot never sends a whole website, Figma file or screenshot to the AI. It sends grouped issue evidence as text, at most about 20,000 input tokens per request. The binding AI limits are requests per minute and per day, and on the free tier, Google's data-use terms.

Figma's limit depends on the seat in the file's plan

With a View/Collab seat, file reads are capped at "up to 20 per month" on every plan. The limit belongs to the plan where the file lives. Buying our own Figma plan does not raise the limit for a client's file (such as Ascendion's).

Most of the analysis does not use AI at all

Seven deterministic scanners produce the findings. AI only rewrites issue text, and that text is checked against the evidence. When AI is unavailable, the scan still completes with the scanners' wording.

AI cost per scan is cents

The 24 October scan (17 pages, 75 issues) used about 130,000 input and 46,000 output tokens. On paid tiers that is about $0.04 (Claude Haiku 5.5) to $0.72 (Claude Sonnet 5.5) per scan. Gemini 3.6 Flash is about $0.27.

Our own code sets the hard ceilings

At most 500 pages per scan, 300 checked links, 200 AI-explained issues, one Figma link per project and one merged design reference. These are design choices, not subscription limits, and only code changes raise them.

Spend on architecture before subscriptions

The highest-value next steps are code: reuse AI text for issues already seen, a provider fallback chain, per-template design references, scan resume and a usage/cost dashboard. Paid AI is only needed for client data (Google's free-tier terms) and higher daily volume.

2. Figma capacity and limitations

Figma has three kinds of limit: what one file can hold, how often the API may be called, and what a subscription plan allows. On top of these, QA Copilot adds its own processing rules. These are often confused, so the table separates them.

LIMIT	VALUE	TYPE	SOURCE	WHAT IT MEANS FOR QA COPILOT
Pages per file	Starter team files: max 3 pages. Professional, Organization: as many as needed	`confirmed` plan-dependent	Figma Help Center, "Create and manage pages"	The Ascendion_Dev file has 53 pages, so it can only exist on a paid plan.
File size / complexity	Figma has no fixed page, frame or layer count. A file is limited by the browser tab's memory (Figma staff describe about 2 GB).	`community / staff` soft limit	Figma forum (staff reply)	Large files open slowly in Figma, and the API can return very large responses (whole Ascendion file at `depth=3`: 2.5 MB of structure only).
API: file content (Tier 1: GET file, GET nodes, GET image)	View/Collab seat: **up to 20 per month** on every plan (may be lower under load). Dev/Full seat: 10/min (Starter), 15/min (Professional), 20/min (Organization); Enterprise not clearly listed	`confirmed` hard, seat- and plan-dependent	Figma REST API rate limits (in effect since 17 Nov 2025)	The scarcest resource. Each Figma extraction costs 1 Tier 1 request.
API: metadata (Tier 3: GET file meta, styles, components)	View/Collab up to 10/min. Dev/Full 50, 100 or 150/min by plan	`confirmed`	Same page	QA Copilot uses `meta` to see whether the file changed. An unchanged file costs no Tier 1 request.
Who the limit belongs to	Personal access token: per user, **per plan of the file**. "A file in a Starter plan gets Starter limits even if the user has an Enterprise Full seat."	`confirmed`	Same page	For a client file, only a Dev/Full seat **in the client's plan** raises the limit.
Rate-limit response	HTTP 429 with `Retry-After`, `X-Figma-Plan-Tier`, `X-Figma-Rate-Limit-Type` (low/high), `X-Figma-Upgrade-Link`. Leaky-bucket algorithm; Figma may change limits	`confirmed`	Same page	On 22 Oct, Figma answered `plan: org`, type `low`, Retry-After 159,075 s (44 h). `observed`
Pagination	The file and nodes endpoints do not paginate. Responses are limited with `ids` and `depth`	`confirmed`	Figma file endpoints	QA Copilot splits large files into page batches itself.
Token scopes	Our token has `file_content:read` + `file_metadata:read`. Styles/components endpoints need `files:read` or `library_content:read`	`observed`	HTTP 403 on 22 Oct	Published-styles listings (Tier 3) are not available with the current token.

LIMIT	VALUE	TYPE	SOURCE	WHAT IT MEANS FOR QA COPILOT
Figma links per project	1 file. With `node-id`: that one frame. Without it: the whole file	our app	`parseFigmaUrl`	One URL can cover an entire website design (whole-file read), but see "merged reference" below.
Response size per request	`FIGMA_MAX_RESPONSE_MB` = 50 (1 to 500)	our app	config	A larger whole file is read page by page: page list plus batches of `FIGMA_PAGES_PER_REQUEST` = 10 pages. A batch that is too large is split in halves.
Retries	Up to 3 retries; waits up to 60 s on `Retry-After`. A longer wait stops the read with a checkpoint	our app	config	A monthly cap does not lose work. "Continue reading" fetches only the missing pages.
What is extracted	Text styles (H1–H6, body, link, button), color styles, a Button component's Primary variant, a "Container" frame	our app	`normalize.ts`	Components, spacing, layout grids and Figma Variables are not used. Variables need the Enterprise-only Variables API.
Merged design reference	All extracted values go into **one** reference: one value per role, applied to every scanned page	our app accuracy limit	`upsertDraftFromExtraction`	A whole-website Figma file mixes templates (home H1 vs article H1). See the recommendation below.

Is one Figma URL for the whole website practical?

For reading: yes. QA Copilot can read a 53-page file through one URL, in batches of 10 pages, with resume. That costs about 7 Tier 1 requests the first time and none while the file is unchanged.

For accurate comparison: no, not today. The tool keeps one value per role, so every website page is compared with one blended design. The accurate setup is **one project per template**, each with one frame link (for example T16 → articles, T1 → home), or the per-template design references proposed in section 8.

PLAN (FIGMA PRICING PAGE)	FULL SEAT	DEV SEAT	COLLAB	VIEW	TIER 1 API WITH A DEV/FULL SEAT	TIER 1 WITH VIEW/COLLAB
Starter	Free	–	–	–	10/min	≤20/month
Professional	$16/mo	$12/mo	$3/mo	free	15/min	≤20/month
Organization (annual)	$55/mo	$25/mo	$5/mo	free	20/min	≤20/month
Enterprise (annual)	$90/mo	$35/mo	$5/mo	free	not clearly listed	≤20/month

Prices in USD per seat per month, read from figma.com/pricing on 24 Oct 2026. The pricing page states no numeric REST API limits ("Rate limits vary by seat, plan, and API endpoint"); the API column comes from the rate-limits page. confirmed

3. AI model capacity and limitations

Our current setup: Google Gemini (`gemini-3.6-flash`) on the **free tier**. It uses 2 parallel requests, paced to 5 per minute, with at most 16,000 output tokens and about 20,000 input tokens per request. QA Copilot also supports Anthropic, any OpenAI-compatible API and Groq, so switching provider is a configuration change.

PROVIDER / MODEL	INPUT $/1M	OUTPUT $/1M	BATCH	CONTEXT	RATE LIMITS	NOTES
Gemini 3.6 Flash (current)	0.75 → 1.50	3.75 → 7.50	50% off	1M class `confirmed` (3.8 Flash: 1,048,576 in / 65,536 out)	Free tier: 5 RPM `observed`; per-day limit unpublished, about 20/day `observed`. RPM, TPM and RPD are per project; RPD resets at midnight Pacific	Prices double on 1 Jan 2027. Free-tier content is used to improve Google products. EEA, CH and UK apps must use paid.
Gemini 3.1 Flash-Lite	0.25	1.50	50% off	large	Paid tiers 1–3 by cumulative spend ($0 / $100 / $1,000)	Cheaper option for simple rule groups.
Gemini 3.1 Pro Preview	2.00 (≤200k)	12.00	50% off	large	No free tier	Strong reasoning; preview.
Claude Haiku 5.5	0.10 (≤100k)	0.50	50% off	see note	Start tier: 1,000 RPM, 2M input TPM, 400k output TPM. Monthly spend cap $500 (Start) / $1,000 (Build) / $200,000 (Scale)	The cheapest capable paid option. Cached input does not count toward input TPM.
Claude Sonnet 5.5	2.00	10.00	50% off	1M at standard price	Same tier table as Haiku 5.5	For complex design/content reasoning.
Claude Opus 5.5	4.00	20.00	50% off	1M at standard price	Same tier table	Not needed for issue explanations.
OpenAI gpt-6-luna	0.10	0.50	50% off	–	Per account tier (not read)	Via the OpenAI-compatible adapter.
OpenAI gpt-6.1-sol	2.00	10.00	50% off	–	Per account tier (not read)	–
Groq openai/gpt-oss-120b	0.15	0.60	–	131,072 / 65,536 out	Developer plan base: 30 RPM, 1K RPD, **8K TPM**, 200K TPD. Free plan not readable	8K TPM is smaller than one of our requests today (on average about 11k in + 4k out on 24 Oct), so requests must be made smaller.

All prices are USD per 1M tokens, standard tier, read on 24 Oct 2026. Gemini 3.6 Flash shows "current → from 1 Jan 2027". `confirmed` unless tagged otherwise.

What drives AI usage in QA Copilot

INPUT	SENT TO AI?	EFFECT ON AI USAGE
Number of website pages	No (indirectly)	More pages mean more findings, but findings are grouped into issues by rule. The 24 Oct scan turned 846 findings into 75 issues, so usage grows much slower than page count.
Figma design	No	Read by deterministic code into design values. Its size affects Figma requests, not AI.
Screenshots	No	Stored as evidence (about 66 MB per 17-page scan) and never sent to AI.

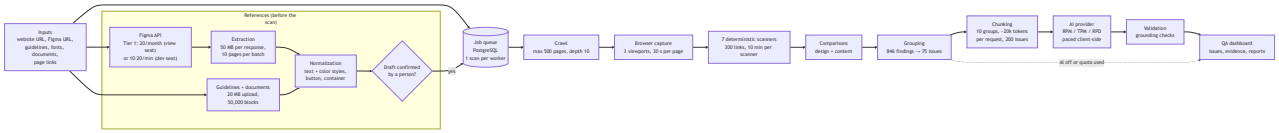
INPUT	SENT TO AI?	EFFECT ON AI USAGE
Brand guidelines	Optional, capped	A deterministic reader extracts values. An AI pass over guideline text is capped at 300 lines / 40,000 characters.
Font files	No	Validated by code.
Content documents	Only relevant sections	Split into sections of about 600 tokens. Only sections matching an issue are attached, each cut to 800 characters.
Issue evidence	Yes	Up to 10 issue groups per request, 5 findings per group, values trimmed, about 20,000 input tokens per request, 200 issues per scan.

What happens at a limit

- **Input too large for one request:** batches are cut to fit before sending. An issue that still does not fit keeps the scanners' wording, with the reason recorded. `our app`
- **Per-minute limit (HTTP 429):** client-side pacing waits for a free slot. The `Retry-After` wait is honoured up to 60 s, with 3 retries. `our app`
- **Daily quota used up:** detected from Google's structured quota ids. The remaining issues keep the scanners' wording, and the AI step says so. Nothing is lost, but there is no automatic switch to another provider. `our app`
- **Provider overloaded or down (HTTP 503, timeouts):** retried. On 24 Oct, 3 of 12 requests still failed with HTTP 503 "high demand", so 19 issues used the scanners' wording. `observed`
- **AI says something the evidence does not support:** the grounding check replaces or rejects it. On 24 Oct, 843 of 867 checks passed and 16 issues had unsupported parts replaced. `observed`

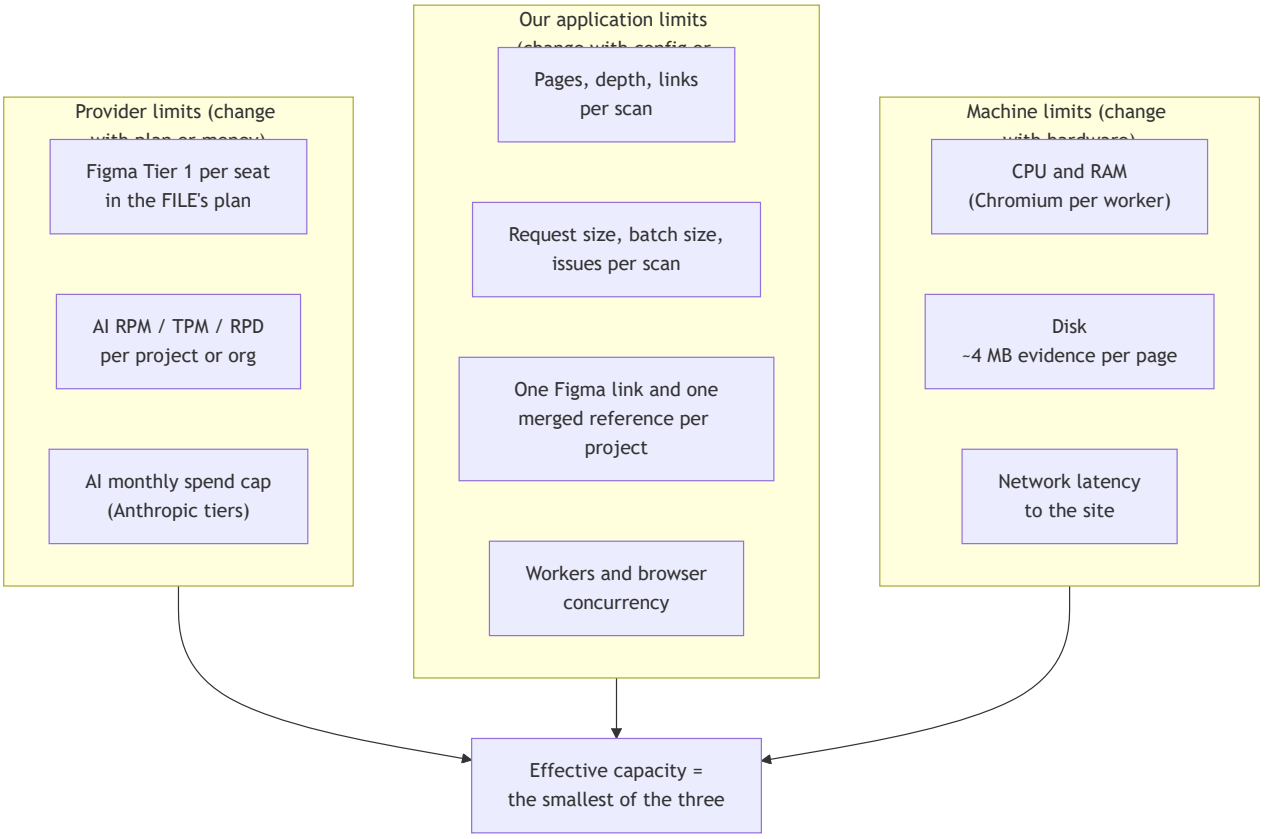
4. Architecture diagrams and failure / recovery flows

4a. End-to-end workflow with the limit at each stage



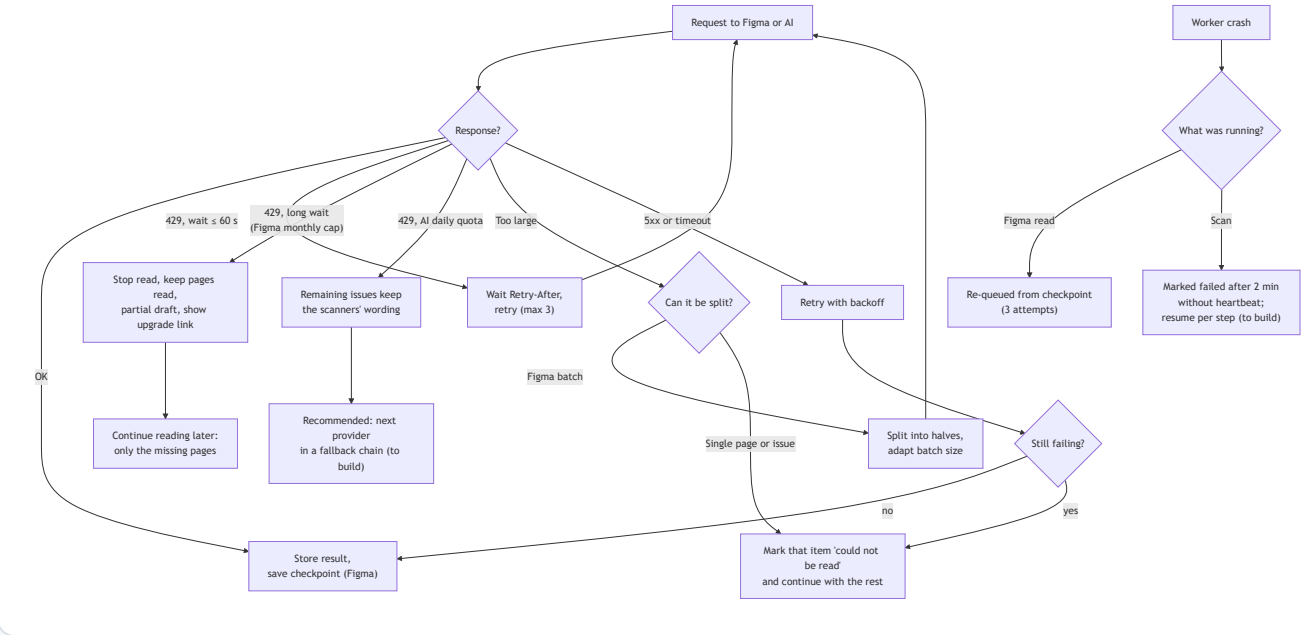
> In plain words: Figma and the documents are read once and turned into design values that a person confirms. The scan then crawls the site, captures each page in a real browser and runs the scanners, all without AI. AI only enters at the end, to explain the grouped issues, and the dotted line shows that the dashboard still works without it.

4b. Three layers of limits, and which one you can change



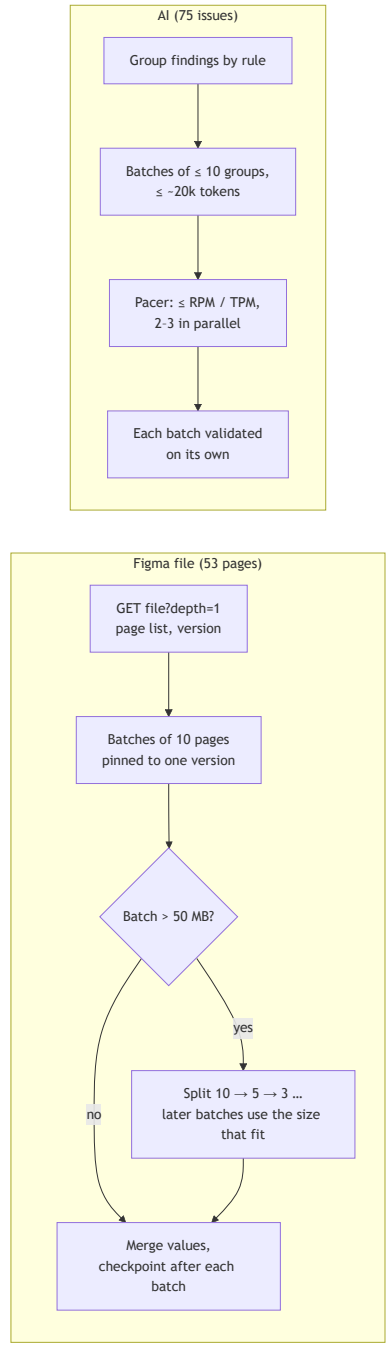
> Capacity is set by whichever layer runs out first. A bigger AI context window changes none of these numbers. Upgrading our own Figma plan does not help with a client's file. Raising `SCAN_MAX_PAGES` does not make a slow machine faster.

4c. What happens when a limit is reached



> Every limit leads to a recorded state, never a silent gap: waited and retried, stopped and resumable, or marked "could not be checked" with the reason. The two red-flag gaps today are the missing provider fallback and the missing scan resume. Both are in the roadmap.

4d. Batching: why small pieces are more reliable



> A failure in one small piece costs only that piece. A 53-page Figma file is never one fragile 200 MB request, and one AI timeout affects 10 issues, not the whole report. Batches also fit inside per-minute limits, which keeps throughput steady instead of stop-start.

5. Scenario analysis

Assumptions `estimate` : the timings scale linearly from the 24 Oct scan (17 pages, 3 viewports, 16 min). AI usage scales with issues at about 1,730 input and 610 output tokens per issue (measured). Issue counts grow slower than pages because findings are grouped. Server costs are excluded. The AI cost uses paid Gemini 3.6 Flash (current price) and Claude Haiku 5.5, per scan.

1. Small: 5 pages, simple Figma

Approach	1 project, 1 Figma frame link, defaults
Time	~6 min `estimate`
AI	~40 issues → 4 requests, ~70k in / 25k out
AI cost	Flash ~$0.14 · Haiku 5.5 ~$0.02 · free tier fits
Bottleneck	None

2. Medium: 25 pages, several components

Approach	1–2 projects (one per template)
Time	~25 min
AI	~85 issues → 9 requests, ~150k in / 52k out
AI cost	Flash ~$0.31 · Haiku ~$0.04 · free tier: about 1 scan/day
Bottleneck	Free-tier requests per day

3. Large: 100 pages, complex Figma

Approach	Project per template, frame links, max pages 100
Time	~1.5 h at current concurrency; ~35 min with `SCAN_CONCURRENCY` 8 on a strong machine
AI	~130 issues → 13–14 requests, ~225k in / 80k out
AI cost	Flash ~$0.47 · Haiku ~$0.06 · free tier: often stops partway
Bottleneck	Capture time, accessibility time, free AI quota, ~400 MB evidence

4. Enterprise: 500+ pages, several Figma files

Approach	Split by section into several projects (the 500-page cap is per scan); several workers
Time	~7 h for 500 pages on one worker; ~1–2 h spread over 4–6 workers
AI	200 issues per scan (cap) → 20 requests, ~350k in / 120k out
AI cost	Flash ~$0.72 · Haiku ~$0.10 per scan; paid tier required
Bottleneck	Our caps (500 pages, 300 links, 200 AI issues), disk (~2 GB per scan), Figma reads for several files

5. Several client sites at once

Approach	`WORKER_CONCURRENCY` up to 8 per process, more worker processes
Bottleneck	AI pacing is per process, so N workers can send N× the RPM and hit 429s. Each client's Figma file is limited by the seat in that client's plan.
Needed	A shared rate limiter across workers (to build), per-client quota tracking

6. A Figma file too large for one request

Handled today	Read page by page (10 per batch, split on "too large", version-pinned, checkpointed). A single page over 50 MB is marked failed and the rest continue.
Better	Link the frames that matter (1 read) instead of the whole file

7. AI quota reached during a scan

Handled today	Scan completes. The remaining issues use the scanners' wording and the AI step reports it (seen on 22 Oct).
Better	Provider fallback chain (for example Gemini → Claude Haiku 5.5), and reuse of AI text for issues already seen

8. Figma throttled or incomplete data

Handled today	Short waits retried. Long waits (44 h seen) stop the read with a checkpoint and partial draft. Missing pages are listed, and unchanged files are not re-read.
Better	A Dev seat in the client's plan, or a copy of the needed frames in our own Professional team

6. Subscription and cost comparison

UPGRADE	COST	WHAT IT ACTUALLY RAISES	WHAT IT DOES NOT CHANGE
Figma Dev seat, our own Professional team	$12/mo	Tier 1 to 15/min, **for files in our team**	Client files in the client's plan (still ≤20/month with a view seat there)
Figma Dev seat in the client's Organization plan	$25/mo (annual), paid by or arranged with the client	Tier 1 to 20/min for that client's files	Our one-link, one-reference-per-project limits
Gemini paid Tier 1 (link billing)	Pay per token: ~$0.27 per 17-page scan; price doubles 1 Jan 2027	Higher RPM/RPD (numbers shown in AI Studio, not published). Client data no longer used to improve Google products	Our 20k-token requests and 200-issue cap; 503 "high demand" can still happen
Anthropic API (Start tier)	Pay per token: Haiku 5.5 ~$0.04, Sonnet 5.5 ~$0.72 per scan; spend cap $500/mo	1,000 RPM and 2M input TPM, enough for many parallel scans	Spend cap (Build $1,000, Scale $200,000); an unrecognized spend-cap 429 has no `retry-after`
Groq Developer	gpt-oss-120b $0.15 / $0.60 per 1M	Cheap tokens	8K TPM: requests must be made smaller and scans take longer
Larger AI context window (any provider)	–	Nothing that matters to us	RPM, RPD, TPM and monthly totals: separate limits

Estimated monthly AI cost `estimate`

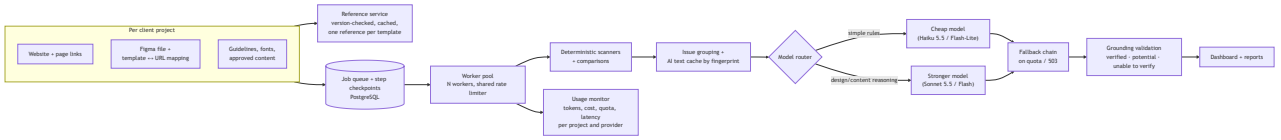
USAGE	ASSUMPTION	GEMINI 3.6 FLASH	GEMINI 3.1 FLASH-LITE	CLAUDE HAIKU 5.5	CLAUDE SONNET 5.5
Small	20 scans × 25 pages	~$6	~$2.40	~$0.80	~$16
Medium	100 scans × 100 pages	~$47	~$18	~$6	~$124
Large	500 scans × 500 pages	~$360	~$135	~$50	~$955

Based on measured tokens per issue and the issue estimates in section 5, at standard (non-batch) prices. Batch APIs halve these figures for non-urgent scans. Figma seats, servers and storage are extra and not included.

7. Optimization strategies, ranked by impact and effort

#	STRATEGY	LIMIT IT SOLVES	BENEFIT	EFFORT	TRADE-OFF	STATUS
1	Deterministic scanners for accessibility, SEO, security, links, performance and responsive	AI cost and accuracy	All findings come from code; AI only explains them	–	–	built
2	Group findings into issues before AI	AI requests	846 → 75 (11× fewer items)	–	–	built
3	Figma version check + frame links + page batches + checkpoint	Tier 1 quota	0 requests for unchanged files	–	–	built
4	**Reuse AI text for issues already seen** (same fingerprint, same evidence)	RPD, cost	Re-scans need AI only for new or changed issues; often 50–90% fewer requests estimate	Low–medium	Changed evidence must invalidate the cache	to build
5	**Provider fallback chain** on daily quota or 503	Availability	No issue left unexplained because one provider is busy	Medium	Mixed wording style; second key to manage	to build
6	**Per-template design references** (Figma frame ↔ URL pattern)	Accuracy (one merged reference)	Fewer false design mismatches; one project covers several templates	Medium–high	More setup per project	to build
7	**Usage and cost dashboard** (tokens, latency, quotas per project)	Visibility	Cost per project, early warning before quotas run out	Low (data already in `ai_run`)	–	to build
8	Model routing: cheap model for simple rules, stronger model for design/content reasoning	Cost	Most issues handled at Haiku/Flash-Lite prices	Medium	Two models to evaluate	to build
9	Scan resume per step after a crash or restart	Reliability on long scans	A 7-hour scan no longer restarts from zero	Medium–high	More checkpoint data	to build
10	Shared rate limiter across workers (in PostgreSQL)	Multi-client concurrency	Many workers stay within one provider limit	Medium	Small latency per request	to build
11	Batch APIs for non-urgent scans	Cost	50% cheaper AI (all three providers)	Medium	Results in minutes to hours, not live	to build
12	Screenshot compression (WebP), deduplication, retention	Disk (~4 MB/page)	3–5× less storage estimate	Low	Slightly lower image quality	to build (garbage collection exists)
13	Client-side pacing, retries with backoff, request-size limits	429 errors	Steady throughput on free tiers	–	–	built

8. Recommended architecture



> The current design already meets most of the requirements:
>
> - multi-project support, all six input types and evidence-backed findings;
> - the three verdict states (confirmed, potential, unable to verify);
> - a provider abstraction, Figma resume and token logging.
>
> Five additions close the gaps:
>
> - per-template references;
> - an AI text cache;
> - a model router with a fallback chain;
> - step-level scan resume;
> - a shared rate limiter with a cost dashboard.
>
> None of them needs a new subscription.

9. Prioritized roadmap

Immediate (days, config or small code)

1. Move client scans to a **paid AI tier** (Gemini Tier 1 or Claude Haiku 5.5). Under Google's terms, free-tier content may be used to improve products and read by human reviewers.
2. Usage and cost dashboard from the existing `ai_run` data.
3. Reuse AI text for issues already seen in earlier scans.
4. Link Figma **frames**, one project per template; avoid whole-file reads.
5. Store the second provider's key; set `AI_REQUESTS_PER_MINUTE` to the paid tier's limit.

Medium term (weeks)

1. Provider fallback chain on daily quota and 503.
2. Per-template design references (frame ↔ URL pattern).
3. Model routing (cheap vs strong).
4. Screenshot compression and retention policy.
5. Agree Figma access with clients: a Dev seat in their plan, or frames copied to our team.

Enterprise scale (months)

1. Step-level scan resume and retry of single steps.
2. Shared, database-backed rate limiter for many workers.
3. Batch API mode for overnight scans (50% cheaper).
4. Per-client quotas and priority queues.
5. Splitting sites over 500 pages into section projects automatically.

10. Decision matrix: subscription, code, or both

LIMITATION	SUBSCRIPTION	CODE / CONFIG	VERDICT
Figma ≤20 reads/month (view seat)	Dev/Full seat **in the file's plan**	Caching and frame links (built); reuse across projects	**Both**: code first; a seat only for clients with frequent design changes
AI requests per day (free tier)	Paid tier	AI text cache, fallback chain	**Both**, and paid is needed anyway for client data
AI per-minute 429s	Higher tier	Pacing (built), shared limiter	**Code** first
AI provider outage / 503	No	Fallback chain	**Code**
Context window	No (not binding)	Chunking (built)	**Nothing** to do
500 pages, 300 links, 200 AI issues per scan	No	Config / code	**Code**
One merged design reference	No	Per-template references	**Code**
Long scans restart after a crash	No	Step resume	**Code**
Many clients in parallel	Higher AI tier at large scale	Workers, shared limiter	**Both**
Monthly AI spend cap (Anthropic)	Tier increase	Spend monitoring, cheap-model routing	**Both**
Disk for evidence	Storage / hosting	Compression, retention	**Code** first

Unknowns, and conditional recommendations

- **Gemini free-tier daily limit:** not published by Google; about 20/day observed. *If* a scan needs more than the day's remainder, the rest of the issues keep the scanners' wording. Check AI Studio → Rate limits.
- **Gemini paid-tier RPM/RPD numbers:** shown only in AI Studio per project. Read them after linking billing, then set `AI_REQUESTS_PER_MINUTE` to match.
- **Figma Enterprise Tier 1 values** and **Groq free-plan values** could not be read reliably from the official pages; check the account consoles.
- **Client Figma access:** unknown whether Ascendion would grant a Dev seat. *If not*, copy the needed frames into our own team (a $12/mo Dev seat gives 15/min) or keep the view seat and rely on caching.
- **Hosting costs** (CPU, RAM, disk for 500-page scans) depend on where QA Copilot is deployed and are not included.
- **Issue growth beyond 17 pages** is estimated, not measured. A 100-page test scan would confirm the scenario numbers.

Sources (read 24 October 2026)

Figma REST API rate limits: tiers, seats, plans, 429 headers (limits in effect since 17 Nov 2025)

Figma pricing: seat prices per plan

Figma Help Center: Create and manage pages (3 pages per file on Starter)

Figma forum: file memory (2 GB per browser tab, staff reply)

Gemini API pricing (page updated 22 Oct 2026)

Gemini API rate limits and usage tiers (page updated 15 Oct 2026)

Gemini API terms: free-tier data use

Anthropic Claude pricing

Anthropic rate limits and spend caps

OpenAI API pricing

Groq rate limits (Developer plan base limits)

Groq models: context, completion limits, prices

Our measurements: QA Copilot database, Ascendion scans of 22 Oct (project Ascendion_Test) and 24 Oct (Ascendion_Testing_2); `packages/core/src/config.ts`; `LLM_PROVIDER_SETUP.md`; `FIGMA_INTEGRATION.md`.

Prices and limits change without notice. Recheck the linked pages before budgeting. Estimates are marked as such, and their assumptions are stated in section 5.